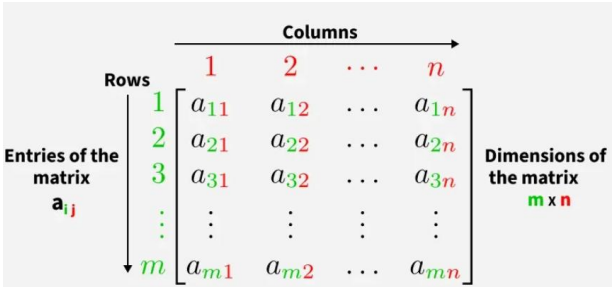
MATRIX — Definition

Matrices are used in machine learning to handle data in multi-dimensional form, making computations efficient.

- A matrix is a two-dimensional array of numbers arranged in rows and columns
- Each element is represented as a[i][j] where i is the row number and j is the column number
- The order of a matrix is written as m×n, where m is the number of rows and n is the number of columns

For example: $A = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \end{bmatrix}$
This matrix has 2 rows and 3 columns. So the order of this matrix is (2 x 3).

General form $A = [a_{ij}]$, where a_{ij}represents the element in the i^{th}row and j^{th}column.



A dataset can be represented as a matrix: $X = \begin{bmatrix} 25 & 50000 & 3 \\ 30 & 65000 & 4 \\ 35 & 80000 & 5 \end{bmatrix}$
where rows represent **observations** and columns represent **features**.

In Python the NumPy library is commonly used to work with matrices. Matrices can be created using the numpy.array() function:

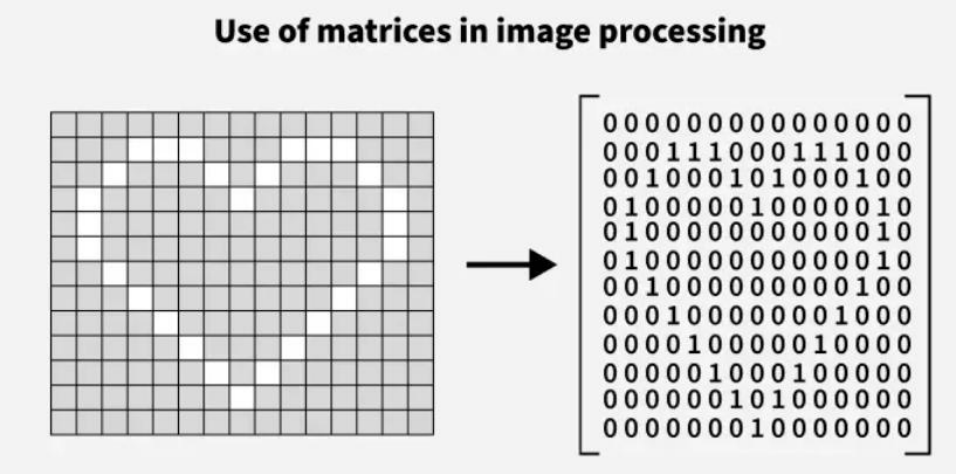
```
import numpy as np
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(matrix)
```

Matrices are:

Data Storage: Acts like a spreadsheet where rows represent individual data samples (instances) and columns represent features or attributes.

Real-World Translation: Digital images are stored as matrices of pixel brightness or color values, and text is converted into embedding matrices



Key Uses in Artificial Intelligence

- **Neural Networks**: Storing connection weights between layers and processing entire batches of data simultaneously using optimized hardware like GPUs.
- **Computer Vision**: Representing pixel grids and applying geometric transformations like rotation, scaling, and translation.
- **Natural Language Processing**: Converting words and sentences into dense semantic vector spaces via embedding matrices.
- **Performance Evaluation**: Utilizing specialized matrices like the Confusion Matrix

TYPES OF MATRICES

Type of Matrix	Definition / Property	Example	AI / ML Relevance
1. Row Matrix	Has only **one row**	$\begin{bmatrix}1 & 2 & 3\end{bmatrix}$	Represents a single data record or feature vector
2. Column Matrix	Has only **one column**	$\begin{bmatrix}1\\2\\3\end{bmatrix}$	Represents an input vector or feature vector
3. Rectangular Matrix	Number of rows ≠ number of columns	$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix}$	Common representation of datasets
4. Square Matrix	Number of rows = number of columns	$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$	Used in transformations, eigenvalue problems and many ML algorithms
5. Zero / Null Matrix	All elements are zero	$\begin{bmatrix}0 & 0\\0 & 0\end{bmatrix}$	Represents absence of values or zero transformation
6. Diagonal Matrix	All non-diagonal elements are zero	$\begin{bmatrix}2 & 0 & 0\\0 & 5 & 0\\0 & 0 & 3\end{bmatrix}$	Useful for scaling and mathematical transformations

7. Identity Matrix	Diagonal elements are 1 and all others are 0	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	Used in inverse operations and transformations
8. Symmetric Matrix	$A = A^T$	$\begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix}$	Important in covariance matrices and optimization
12. Singular Matrix	Determinant is zero: $\det(A) = 0$	$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$	Indicates dependent information/features
13. Non-Singular / Invertible Matrix	Determinant is non-zero: $\det(A) \neq 0$	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	Can be inverted; useful in solving mathematical models

Addition, Subtraction and Scalar Multiplication of Matrices

Matrix addition/ subtraction combines two matrices of the same order by adding / subtracting their corresponding elements.

For addition and Subtraction the two matrices must have same order.

Definition:

The **matrix sum of *A* and *B*** is obtained by adding corresponding entries of *A* and *B*,

$$(a + b)_{ij} \overset{def}{=} a_{ij} + b_{ij}$$

The **matrix difference of *A* and *B*** is obtained by subtracting the entries of *B* from the corresponding ones in *A*,

$$(a - b)_{ij} \overset{def}{=} a_{ij} - b_{ij}$$

Example 1

Add the matrices $A = \begin{pmatrix} 0 & -3 \\ 5 & 2 \end{pmatrix}$ and $B = \begin{pmatrix} 13 & 1 \\ -4 & 5 \end{pmatrix}$.

As the matrices have the same order, they can be added.

$$A + B = \begin{pmatrix} 0 & -3 \\ 5 & 2 \end{pmatrix} + \begin{pmatrix} 13 & 1 \\ -4 & 5 \end{pmatrix}$$

$$= \begin{pmatrix} 0 + 13 & -3 + 1 \\ 5 + (-4) & 2 + 5 \end{pmatrix}$$

$$= \begin{pmatrix} 13 & -2 \\ 1 & 7 \end{pmatrix}$$

Matrix addition is used when **combining or updating feature representations**, such as adding bias or residual information in neural networks.

Example 2

Find the difference $P - Q$ where $P = \begin{pmatrix} 4 & -2 & 0 \\ 1 & 0 & 37 \end{pmatrix}$ and $Q = \begin{pmatrix} -4 & -2 & 2 \\ -8 & 3 & 6 \end{pmatrix}$.

$$P - Q = \begin{pmatrix} 4 & -2 & 0 \\ 1 & 0 & 37 \end{pmatrix} - \begin{pmatrix} -4 & -2 & 2 \\ -8 & 3 & 6 \end{pmatrix}$$

$$= \begin{pmatrix} 8 & 0 & -2 \\ 9 & -3 & 31 \end{pmatrix}$$

Scalar Multiplication of a Matrix

Scalar multiplication scales every individual entry of a matrix by a single real or complex number. The order of the matrix does not change.

The **scalar multiplication of** A **by** k ($k \in \mathbb{R}$) is obtained by multiplying each entry of A by k,

$$(ka)_{ij} \overset{def}{=} ka_{ij}$$

If $A = \begin{pmatrix} 4 & -4 & 0 \\ 5 & 6 & 1 \\ 8 & 9 & -1 \end{pmatrix}$, calculate $\frac{1}{2}A$.

$$\frac{1}{2}A = \frac{1}{2}\begin{pmatrix} 4 & -4 & 0 \\ 5 & 6 & 1 \\ 8 & 9 & -1 \end{pmatrix}$$

$$= \begin{pmatrix} 4/2 & -4/2 & 0/2 \\ 5/2 & 6/2 & 1/2 \\ 8/2 & 9/2 & -1/2 \end{pmatrix}$$

$$= \begin{pmatrix} 2 & -2 & 0 \\ \frac{5}{2} & 3 & \frac{1}{2} \\ 4 & \frac{9}{2} & -\frac{1}{2} \end{pmatrix}$$

Scaling is important in Feature Scaling, Normalization, Weights updates, Optimization Algorithm.

- **Distribution over matrix addition:** $k(A + B) = kA + kB$
- **Distribution over scalar addition:** $(k + h)A = kA + hA$
- **Associative with scalars:** $k(hA) = (kh)A$

- **Identity element:** $1 \cdot A = A$
- **Zero property:** $0 \cdot A = 0$ (results in a zero matrix)

Transpose of a Matrix

Tranpose of a matrix means interchange of the rows and columns, i.e. rows becomes columns and columns becomes rows. Ex:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$ Remember $(AB)^T = B^T A^T$

Transpose is frequently used in: PCA, Covariance calculations, Optimization, Machine-Learning Algorithms.

Matrix Multiplication

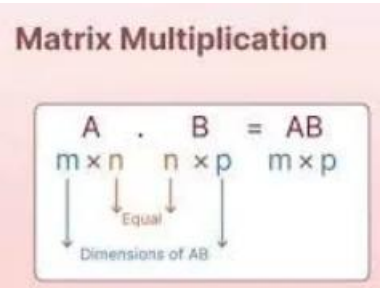
Matrix multiplication combines two matrices using a row-by-column dot product rule. It is **not element-by-element multiplication**.

For two matrices $A$and B:

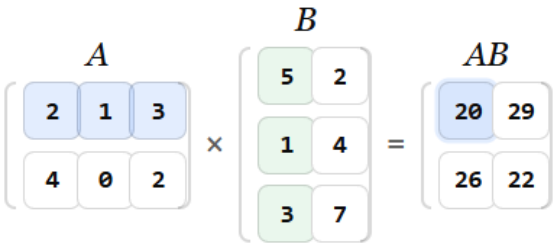
$$A_{m\times n} \times B_{n\times p} = C_{m\times p}$$

Important rule: Inner dimensions must match

$$\boxed{(m \times n)(n \times p) = (m \times p)}$$



To calculate AB, take **each row of A × each column of B**.



For example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

$$AB = \begin{bmatrix} 1(5)+2(7) & 1(6)+2(8) \\ 3(5)+4(7) & 3(6)+4(8) \end{bmatrix} \quad AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Remember, **Row × Column → Add the products**

Properties of Matrix Multiplication

Property	Formula	Meaning
Closure	$AB = C$	Result is also a matrix
Associative	$(AB)C = A(BC)$	Grouping does not matter
Distributive	$A(B + C) = AB + AC$	Multiplication distributes over addition
Scalar distribution	$k(A + B) = kA + kB$	Scalar distributes over matrix addition
Identity	$AI = IA = A$	Identity matrix does not change A
Not commutative	$AB \neq BA$ generally	Order usually matters

Why Matrix multiplication is used in AI?

Matrix multiplication is fundamental to **neural networks**. For example, a neural network layer essentially performs: $\boxed{Y = WX + b}$
where:

- X= input data
- W= weight matrix
- b= bias
- Y= output

So matrix multiplication helps in building the mathematical foundation for understanding **neural networks and deep learning.**

Q1. Multiply

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Q2 Multiply

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

Q3 Multiply

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \end{bmatrix}$$

Hadamard Product:

The **Hadamard product**(also referred to as the **element-wise product**) is the element-wise multiplication of two tensors or matrices of the same shape, where each element in the resulting matrix is the product of corresponding elements from the original matrices.

The two matrices have the same dimensions. If A and B are both of size $m \times n$, then their Hadamard product $C = A \circ B$ is also of size $m \times n$, where each element is computed as follows:

$$c_{ij} = a_{ij} \cdot b_{ij}$$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$A \odot B = \begin{bmatrix} 1 \times 5 & 2 \times 6 \\ 3 \times 7 & 4 \times 8 \end{bmatrix} \qquad A \odot B = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

⭐ Very simple AI example: Image masking

Suppose an image is represented as:

$$Image = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

and we want to keep only the left side:

$$Mask = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

Hadamard product:

$$Image \odot Mask = \begin{bmatrix} 10 \times 1 & 20 \times 0 \\ 30 \times 1 & 40 \times 0 \end{bmatrix} = \begin{bmatrix} 10 & 0 \\ 30 & 0 \end{bmatrix}$$

Meaning: `1 = keep` and `0 = remove`.

Use of Hadamard product in AI

Neural Networks	Element-wise multiplication is used in different operations inside neural networks	Applying a gate to neural activations
LSTM/GRU	Very important for **gates** that decide what information to keep or forget	`new memory = gate ⊙ candidate memory`
Computer Vision	Used with masks to keep or suppress particular pixels/features	Image × mask
Attention mechanisms	Can be used for element-wise feature weighting	Feature × attention weights
Feature selection	A mask can select important features	Features × 0/1 mask
Deep Learning	Used for element-wise operations between activation/feature matrices	Activation × weight/mask
Optimization	Used in some gradient and parameter-wise operations	Parameter × update/mask

FUNCTIONS IN MATRICES

In linear algebra and matrix analysis, "functions in matrices" usually refers to **matrix functions**: a way to apply a scalar function f(like e^x, $\sin x$, $\log x$) to a square matrix Aalso produce another matrix $f(A)$.

There's also the related idea that **matrices themselves represent functions** (linear maps) via $x \mapsto Ax$.

1. Matrices as functions (linear maps)

- Every matrix represents a Linear function that maps vectors from one space to another.
  Identity matrix defines the identity function.
  Zero matrix defines the zero function.
  Inverse matrix defines the inverse function.

  If you multiply a fixed matrix A by a vector x, it creates a **function T(x) = Ax** that rotates, scales, or reflects vectors in space.

  Example: Let $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$, $x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$.

  Then the linear function $T_A(x) = Ax$ gives

  $$Ax = \begin{pmatrix} 2 \cdot 1 + 1 \cdot 2 \\ 0 \cdot 1 + 3 \cdot 2 \end{pmatrix} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}.$$

  So the matrix A "acts as" the function that maps $(1, 2) \mapsto (4, 6)$

- **(Identity matrix):** Each real-valued, Euclidean vector space $\mathbb{R}^n$is associated with an **identity matrix**, denoted $\boldsymbol{I}_{n\times n}$,

$$\boldsymbol{I}_3 := \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

On applying matrix-vector multiplication using an identity matrix $\boldsymbol{I}_n$with any vector $\boldsymbol{x} \in \mathbb{R}^n$will result in the same vector x. So **function $\boldsymbol{T}(\boldsymbol{x}) := \boldsymbol{I}\boldsymbol{x}$ is the identity function with domain $\mathbb{R}^{\boldsymbol{n}}$**

- **(Zero matrix):** Each real-valued, Euclidean vector space $\mathbb{R}^n$is associated with a **zero matrix**, denoted $\mathbf{0}_{n\times n}$. the zero matrix for $\mathbb{R}^3$is defined as

$$\mathbf{0}_3 := \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

On applying matrix-vector multiplication using identity matrix 0_n with any vector $\boldsymbol{x} \in \mathbb{R}^n$, the result will be zero vector. Thus, a **function $\boldsymbol{T}(\boldsymbol{x}) := \mathbf{0}_{\boldsymbol{n}}\boldsymbol{x}$is the zero function for domain $\mathbb{R}^{\boldsymbol{n}}$**

- **(Inverse matrix):** For function T(x) = A(x), let C be a matrix such that function $F(\boldsymbol{x}) = \boldsymbol{C}\boldsymbol{x}$ is the inverse function of T. That is, $F = T^{-1}$where $T^{-1}(T(x)) = x$ If this is the case, then we call $\boldsymbol{C}$ the **inverse matrix** of $\boldsymbol{A}$ **denoted as $\boldsymbol{A^{-1}}$.** On applying Inverse Function, transformation can be reversed.

2. Functions of a matrix: $f(A)$

Here we take a scalar function $f: \Omega \subseteq \mathbb{C} \to \mathbb{C}$and extend it to square matrices $A \in \mathbb{C}^{n\times n}$, producing a new matrix $f(A)$.

2.1 Polynomial functions

If $f(t) = c_0 + c_1 t + \cdots + c_m t^m$is a polynomial, define

$$f(A) = c_0 I + c_1 A + \cdots + c_m A^m.$$

This is straightforward because we can multiply matrices and take powers of A

FOR EXAMPLE:

Take the scalar polynomial $f(t) = t^2 + 2t + 1 = (t+1)^2$.

For the same matrix A, define $f(A) = A^2 + 2A + I$.

Compute:

- $A^2 = (2 \quad 1\,0 \quad 3)^2 = (4 \quad 5\,0 \quad 9)$
- $2A = (4 \quad 2\,0 \quad 6)$
- $I = (1 \quad 0\,0 \quad 1)$

So

$$f(A) = A^2 + 2A + I = \begin{pmatrix} 4 & 5 \\ 0 & 9 \end{pmatrix} + \begin{pmatrix} 4 & 2 \\ 0 & 6 \end{pmatrix} + \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 9 & 7 \\ 0 & 16 \end{pmatrix}.$$

This matches the computed result [[9, 7], [0, 16]].

2.2 Power series

If f has a convergent power series around 0,

$$f(t) = \sum_{k=0}^{\infty} a_k\, t^k,$$

then for suitable matrices A, define

$$f(A) = \sum_{k=0}^{\infty} a_k\, A^k = a_0 I + a_1 A + a_2 A^2 + \cdots .$$

2.3 Matrix exponential e^A

The matrix exponential is defined by the series

$$e^A = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \cdots .= \sum_{k=0}^{\infty} \frac{A^k}{k!}.$$

Common matrix functions

Some standard examples:

- **Matrix exponential**: $e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!}$.

  Matrix logarithm: $\log(A)$, defined when A has no eigenvalues on the closed negative real axis (for the principal branch).
- **Matrix powers**: A^α for real or complex α, defined via eigen-decomposition or other methods. Used in fractional diffusion models, graph methods, etc.
- **Trigonometric functions**: $\sin(A)$, $\cos(A)$, defined via their series or eigenvalue-wise.

4. Why this matters in ML / data science

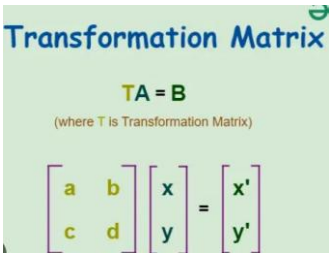
- **Linear layers**: $x \mapsto Wx + b$ are matrix functions in the "matrices as functions" sense.
- **Covariance and kernels**: operations like C^{-1}, $C^{1/2}$, $\log C$ are matrix functions of the covariance matrix.
- **Dynamical systems & RNNs / continuous-time models**: matrix exponentials e^{At} appear in linear dynamical systems and some modern continuous-depth networks.
- **Spectral methods**: applying functions to eigenvalues (e.g., thresholding, regularization) is effectively computing $f(A)$ for symmetric matrices.

LINEAR TRANSFORMATION

A **linear transformation** (or linear map) is a function between vector spaces that maps vectors from one space to another preserving vector addition and scalar multiplication.

It satisfies $T_A(x + y) = T_A(x) + T_A(y)$and $T_A(cx) = cT_A(x)$.

A **transformation matrix** is a square matrix that represents a linear transformation of vectors. It is used to perform operations such as rotation, scaling, reflection, and shearing by multiplying the matrix with a vector. The matrix defines how points or vectors are transformed while preserving the linear properties of the space.



Common 2D transformation formulas include:

- **Scaling (stretch or shrink):**

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

- **Rotation (by angle θ counterclockwise):**

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

- **Reflection (across the x-axis):**

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Common 3D Component Matrices

- **Translation Matrix:**

$$\begin{bmatrix} 1 & 0 & 0 & D_x \\ 0 & 1 & 0 & D_y \\ 0 & 0 & 1 & D_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Scaling Matrix:**

$$\begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

How a 3D Transformation Matrix Works

To transform a 3D point (x, y, z), you add an extra value of `1` to turn it into a 4x1 vector $[x, y, z, 1]^T$. You then multiply this vector by a 4 × 4 matrix: YouTube · Engineering S... +1

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & D_x \\ R_{21} & R_{22} & R_{23} & D_y \\ R_{31} & R_{32} & R_{33} & D_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- **Top-Left 3 × 3 sub-matrix (R):** Handles rotation, scaling, or shearing.
- **Top-Right 3 × 1 column (D_x, D_y, D_z):** Handles translation (movement along X, Y, and Z axes).
- **Bottom row (0, 0, 0, 1):** Used for matrix closure and perspective projection.

The transformation matrix T changes the direction or size of the vector and gives you a new version of it.

1. Translation Matrix

A translation matrix is used to shift objects in a coordinate system.

Example: Let's consider a point P(2, 3) and apply a translation of (4, 1) units.

Sol: *Given point P = (2, 3) and translation vector T = (4, -1), the translation matrix is:*

$$\begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

Applying the translation matrix to point P:

$$\begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 2 \\ 1 \end{pmatrix}$$

Therefore, after the translation, point P(2, 3) is moved to P'(6, 2).

2. Rotation Matrix

A rotation matrix is used to rotate objects in a coordinate system.

Example: Let's rotate a point Q(1, 1) by 90 degrees counterclockwise.
Solution:
Given point Q = (1, 1) and rotation angle θ = 90 degrees, the rotation matrix is:

$$R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

Applying the rotation matrix to point Q:

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

After the rotation, point Q(1, 1) is rotated to Q'(-1, 1).

3. Scaling Matrix

A scaling matrix is used to resize objects in a coordinate system.
Example: Let's scale a rectangle with vertices A(1, 1), B(1, 3), C(3, 3), and D(3, 1) by a factor of 2 in the x-direction and 3 in the y-direction.
Solution:
Given rectangle ABCD and scaling factors sx = 2, sy = 3, the scaling matrix is:

$$S = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$$

Applying the scaling matrix to the vertices of the rectangle:
A'(2, 3), B'(2, 9), C'(6, 9), D'(6, 3)

4. Combined Matrix

A combined matrix applies multiple transformations in sequence.
Example: Let's translate point P(1, 2) to (3, 4) and then rotate it 45 degrees counterclockwise.
Solution:
Given point P = (1, 2), translation vector T = (3, 4), rotation angle θ = 45 degrees, the combined matrix is:
C = R·T
Applying the combined matrix to point P:
First, translate by T:

$$T = \begin{pmatrix} 1 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P' = T \cdot P = \begin{pmatrix} 4 \\ 6 \\ 1 \end{pmatrix}$$

Then, rotate by R:

$$R = \begin{pmatrix} \cos(45) & -\sin(45) \\ \sin(45) & \cos(45) \end{pmatrix}$$

$$P'' = R \cdot P' = \begin{pmatrix} -1 \\ 5 \end{pmatrix}$$

5. Reflection Matrix: A reflection matrix is used to mirror objects across a line or plane.

- **About the x-axis (y = 0):** Keeps x the same and negates y (x' = x, y' = -y).

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **About the y-axis (x = 0):** Keeps y the same and negates x (x' = -x, y' = y).

$$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **About the origin (0, 0):** Negates both x and y (x' = -x, y' = -y).

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **About the line y = x:** Interchanges x and y (x' = y, y' = x).

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- **About the line y = -x:** Interchanges and negates x and y (x' = -y, y' = -x).

$$\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

Example: Let's reflect a point Q(2, 3) across the x-axis.

Solution:

Given point Q = (2, 3), the reflection matrix about the x-axis is:

$R_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Applying the reflection matrix to point Q: $R_x \cdot Q = \begin{pmatrix} 2 \\ -3 \end{pmatrix}$

After reflection, point Q(2, 3) is mirrored to Q'(2, -3).

6. Shear Matrix: A shear matrix is used to skew objects in a coordinate system.

Shear Matrices (2D)

A shear transformation distorts an object's shape like a deck of cards pushed sideways. One coordinate changes based on a factor while the other stays fixed. Scribd +1

- **X-Shear:** Shifts x-values proportional to y using a shear factor sh_x ($x' = x + sh_x \cdot y$, y' = y). Vertical lines tilt.

$$\begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix}$$

- **Y-Shear:** Shifts y-values proportional to x using a shear factor sh_y (x' = x, $y' = sh_y \cdot x + y$). Horizontal lines slope.

$$\begin{bmatrix} 1 & 0 \\ sh_y & 1 \end{bmatrix}$$

Example: Let's shear a rectangle with vertices A(1, 1), B(1, 3), C(3, 3), D(3, 1) in the x-direction by a factor of 2.

Solution:

Given rectangle ABCD and shear factor kx = 2, the shear matrix is:

$H_x = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$

Applying the shear matrix to the vertices of the rectangle:

A'(3, 1), B'(7, 3), C'(9, 3), D'(5, 1)

Example: Given point P = (1, 1), Scaling factor sx = 2, Rotation angle θ = 30 degrees, and Translation vector T = (2, 3)

Solution:

The transformations should be applied in the following order:

A = T.R.S

That means we first scale, then rotate, and finally translate the point.

First, scale by S:

$S = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$

$P' = S \cdot P = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$

Then, rotate by R:

- $R = \begin{pmatrix} \cos(30) & -\sin(30) \\ \sin(30) & \cos(30) \end{pmatrix}$
- $P'' = R \cdot P' = \begin{pmatrix} 1.232 \\ 1.866 \end{pmatrix}$

Finally, translated by T:

- $T = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{pmatrix}$
- $P''' = T \cdot P'' = \begin{pmatrix} 3.232 \\ 4.866 \end{pmatrix}$

After the affine transformation, point P(1, 1).

Example : Find the new matrix after transformation using the transformation matrix $\begin{pmatrix} 2 & -3 \\ 1 & 2 \end{pmatrix}$ on the vector A = 5i + 4j.

Solution:

Given transformation matrix is $T = \begin{pmatrix} 2 & -3 \\ 1 & 2 \end{pmatrix}$

Given vector A = 5i + 4j is written as a column matrix as $A = \begin{pmatrix} 5 \\ 4 \end{pmatrix}$

Let new matrix after transformation be B, and we have the transformation formula as TA = B

B = TA = $\begin{pmatrix} 2 & -3 \\ 1 & 2 \end{pmatrix}$x $\begin{pmatrix} 5 \\ 4 \end{pmatrix}$

B = $\begin{pmatrix} 2*5+(-3)*4 \\ 1*5+2*4 \end{pmatrix}$

B = $\begin{pmatrix} -2 \\ 13 \end{pmatrix}$

B = -2i + 13j

Therefore, the new matrix on transformation is -2i + 13j

Applications of Transformation.

Transformation matrices have **numerous applications** in various fields, including:

- **Computer Graphics:** Used for rendering 3D scenes, modeling objects, and applying transformations to vertices.
- **Image Processing:** Applied for image warping, distortion correction, and geometric transformations.
- **Robotics:** Used to determine the geometric properties and positions of the end-effectors of robotic manipulators.
- **Geometric Modeling:** Plays an important part in both CAD/CAM systems, especially in creating and modifying shapes, surfaces, and solids using parametric representations.
- **Mathematics and Physics:** Applied in the study of linear transformation, vector space, and coordinate systems.

DETERMINANTS

A Determinant is a **scalar value** computed from a **square matrix** that tells us whether the matrix is invertible and whether a system of equations has a unique solution.

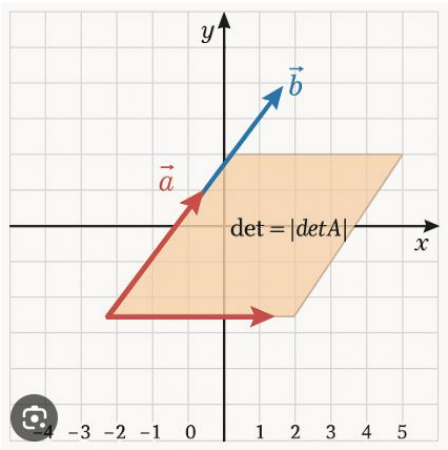
1. What it represents (intuition)

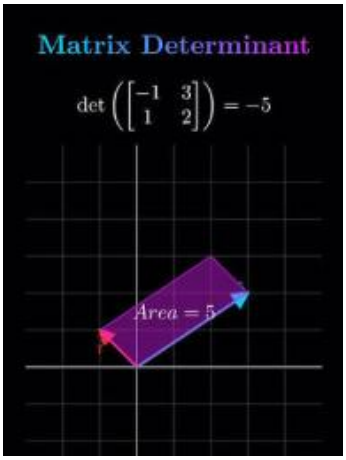
For a matrix Aviewed as a linear transformation $x \mapsto Ax$:

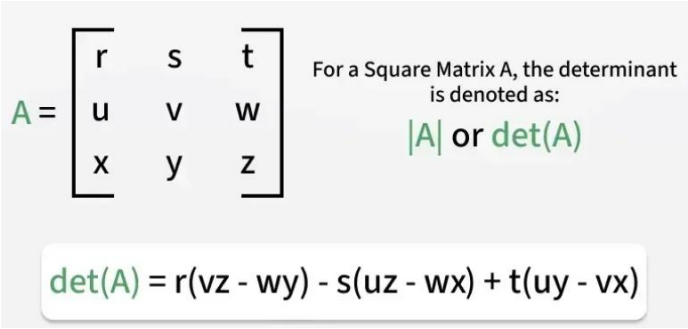
- In 2D: $|\det(A)|$is the **area scaling factor**.
  If you take the unit square ,the area of the resulting parallelogram is $|\det(A)|$.
- In 3D: $|\det(A)|$is the **volume scaling factor** for the parallelepiped spanned by the column vectors of A.
- The **sign** of $\det(A)$tells you about orientation:
  - $\det(A) > 0$: orientation preserved
  - $\det(A) < 0$: orientation reversed (reflection involved)
  - $\det(A) = 0$: the transformation squashes space into a lower dimension (not invertible)

Algebraically, $\det(A) \neq 0$if and only if A is invertible.

- It serves as a scaling factor that is used for the transformation of a matrix.
- It is a single numerical value that plays a key role in various matrix operations, such as calculating the inverse of a matrix or solving systems of linear equations.
- The determinant of a matrix A is denoted by |A| or sometimes as det(A).







Example: Find the Determinant of A = $\begin{bmatrix}3 & 2\\ 2 & 3\end{bmatrix}$.

Determinant of A = $\begin{bmatrix}3 & 2\\ 2 & 3\end{bmatrix}_{2\times2}$ *is calculated as,*

$|A| = \begin{vmatrix}3 & 2\\ 2 & 3\end{vmatrix}$

$|A| = 3 \times 3 - 2 \times 2$
$= 9 - 4$
$= 5$

Key properties (very useful)

For square matrices A, B of the same size and scalar c:

- $\det(AB) = \det(A)\det(B)$
- $\det(A^T) = \det(A)$
- $\det(cA) = c^n \det(A)$ for an $n \times n$ matrix

$\det(A) = 0 \Leftrightarrow A$ is **singular** (not invertible) $\Leftrightarrow$ columns (or rows) are linearly dependent

Application of Determinants in AI

Determinants act as critical scalar indicators in artificial intelligence and machine learning, dictating whether data transformations are reversible, stable, or structurally sound.

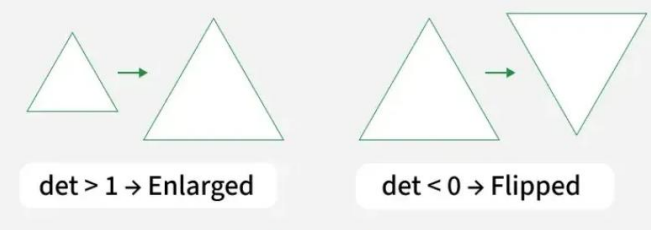
Computer Graphics: Used to perform geometric transformations such as scaling, rotation, and reflection in 2D and 3D graphics.

- They help us to determine how an object's size, shape, and orientation change under transformations.
- **A zero determinant indicates a collapse of shape into a line or point, while a negative determinant shows a mirrored transformation**.

$$\boxed{|\det(A)| = \text{area scaling factor}}$$

det(A)=2, the transformation makes the area **2 times larger**.

If $\det(A) = 0.5$ the area becomes **half**.



Determinants in 2D Transformations: Orientation & Scaling

Cryptography: Ensure that encryption matrices are invertible in techniques like the Hill Cipher.

The Hill cipher encrypts groups of letters using matrix multiplication.

A non-zero determinant guarantees that encrypted messages can be decrypted accurately.

The basic flow is:

- Converts letters into numbers. (from A to Z)

Letter	A	B	C	D
Number	0	1	2	3

- Choose a Encryption key matrix:

$$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$$

- Choose plain text: HELP

Since matrix is 2x2, it takes 2 letters at a time.

```
HELP
 ↓
HE | LP
```

- Convert letters to numbers:

H =7, E = 4 so $P_1 = \begin{bmatrix} 7 \\ 4 \end{bmatrix}$

L=11, P =15 so $P_2 = \begin{bmatrix} 11 \\ 15 \end{bmatrix}$

- Apply Matrix Multiplication:

The encryption formula is: **C = KP (mod 26)** [as there are 26 letters].

Where K = key matrix, P = plaintext vector, C = ciphertext vector

- Encrypt "HE"

$$C = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ 4 \end{bmatrix} = \begin{bmatrix} 33 \\ 34 \end{bmatrix}$$

Now take mod 26:

33 mod 26 = 7

34 mod 26 = 8 convert back to letters 7 = H, 8 = I so,

$$\boxed{HE \rightarrow HI}$$

$$\boxed{HELP \rightarrow HIAT}$$

- The key matrix must be invertible .

For a Hill cipher key matrix, its determinant must satisfy:

$$\boxed{\gcd(\det(K), 26) = 1}$$ We need a number $x$such that: $9x \equiv 1 \pmod{26}$

Find a number which, when multiplied by 9, gives a remainder of 1 when divided by 26. Here: $9 \times 3 = 27 \equiv 1 \pmod{26}$

For our matrix, det(K) =9, and **gcd(9, 26) =1**. So matrix is valid Hill Cipher Key.

So, we can find inverse of matrix K, K^{-1} and decrypt the message.

The matrix inverse exists modulo 26 only when the determinant has a **multiplicative inverse modulo 26**.

So: $\boxed{\det(K) \text{ must be relatively prime to } 26}$

If the determinant has no inverse modulo 26, the key matrix cannot be inverted, and **proper decryption is impossible**.

Consider:

$$K = \begin{bmatrix} 2 & 4 \\ 3 & 6 \end{bmatrix}$$

Its determinant is:

$$(2)(6) - (4)(3) = 12 - 12 = 0$$

Since

$$\det(K) = 0$$

the matrix has **no inverse**.

Therefore, it cannot be used as a valid Hill-cipher encryption key.

Robotics and Control Systems: Applied in robot kinematics to check if transformation matrices are invertible, ensuring precise control over robotic arms.

- They are also used in stability analysis of control systems.

Data Science and Machine Learning: Used in multivariate statistical analysis, especially in probability density functions of the Gaussian distribution.

- They help in matrix inversion for algorithms like linear regression and optimization problems in machine learning.

Computer Vision: Used in image transformation and perspective correction.

- They help ensure transformations are valid and reversible, which is essential in facial recognition, object tracking, and augmented reality.

Network Analysis: In graph theory, determinants of certain matrices (like Laplacian matrices) are used to determine network properties such as the number of spanning trees in a network.

- This has applications in designing reliable communication and transportation networks.

Structural Analysis: Engineers use determinants to solve equilibrium equations in structures like bridges, buildings, and towers.

- It helps in finite element methods for predicting stresses and deformations under various conditions.

IDENTITY MATRIX

- **Definition:** A square matrix (same number of rows and columns) denoted as I or I_n.
- **Structure:** Elements on the main diagonal (top-left to bottom-right) are 1, and all other elements are 0.
- **The "1" of Matrices:** Multiplying any matrix A by I results in the original matrix (A x I = I x A = A)
- The identity Matrix is also known as the **Unit Matrix.**

- The identity matrix I is the square matrix that acts as the multiplicative identity in matrix algebra.

$$I_{2\times2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}_{2\times2} \qquad I_{3\times3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}_{3\times3}$$

Unit matrix of order 2× 2 *Unit matrix of order 3 × 3*

- The **determinant of an identity matrix** or unit matrix is always equal to 1, i.e., $|\boldsymbol{I}| = 1$.
- The **inverse of an identity matrix** or unit matrix is the identity matrix itself. $I^{-1} = I$
- The **product of two inverse matrices** is an identity matrix.

INVERTIBLE MATRICES

An invertible matrix is a square matrix that can be reversed using an inverse matrix (A^{-1}) to undo a linear transformation and restore original data.

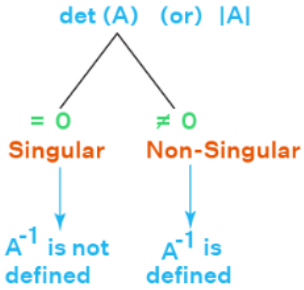
Let A be a square matrix of order n × n. The matrix A is said to be invertible if there exists another matrix B of the same order such that,

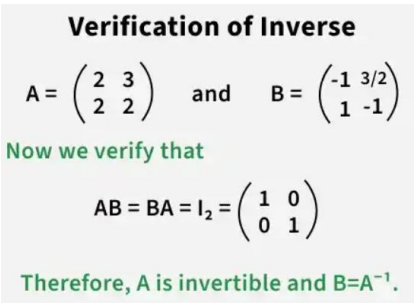
$AB = BA = I_n$

where I_n is the identity matrix of order n × n.

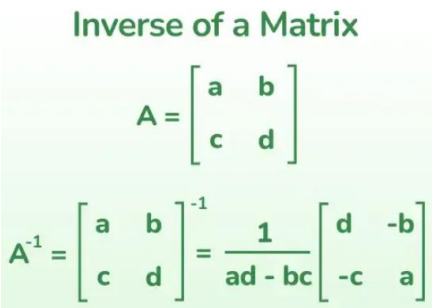
In this case, B is called the inverse of A and is denoted by A^{-1}.

*A matrix is invertible (has an inverse) if and only if its determinant is non-zero (det(A)≠0). If the determinant is zero (det(A)=0), the matrix is **singular** and does not have an inverse.*





$$A^{-1} = \frac{\text{Adj } A}{|A|}$$



What is an Invertible Matrix?

- An invertible matrix, or non-singular matrix, is a square matrix.
- Multiplying the matrix by its inverse gives an identity matrix ($A \cdot A^{-1} = I$).
- It requires a non-zero determinant $\det(A) \neq 0$ and full rank.
- Non-invertible or singular matrices lose information during transformation, meaning they cannot be reversed.

The product AB is invertible and

$$(AB)^{-1} = B^{-1}A^{-1}$$

The inverse of the inverse is the original matrix:

$$(A^{-1})^{-1} = A$$

The determinant of the inverse is the reciprocal:

$$\det(A^{-1}) = 1/\det(A)$$

The transpose of the inverse equals the inverse of the transpose:

$$(A^{-1})^T = (A^T)^{-1}$$

Role in Artificial Intelligence and Machine Learning

An **invertible matrix** is useful when we want to perform a transformation and still be able to **recover the original information**.

Transform → Process → Reverse Transform

An invertible matrix helps solve linear regression parameters, optimize objective functions, and reverse transformations in generative models.

- **Solving Linear Systems:** AI models use linear equations written as Ax=B. Finding $x = A^{-1}B$ helps calculate weights and parameters in regression and classical machine learning.
- **Neural Networks :** use matrix transformations. $Y = WX + b$

  When W is invertible, the transformation can theoretically be reversed:

  $$X = W^{-1}(Y - b)$$

  This idea is important in **invertible neural networks**, where the model is designed to preserve enough information to reconstruct the input.

- **Feature Transformation**

  Suppose original features are: $X = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ and we apply an invertible transformation:

  $$Y = AX$$

Because $A$is invertible, we can recover $\boxed{X = A^{-1}Y}$

This is useful when changing the representation of data while retaining the ability to recover the original representation.

- **Computer Vision**

  Invertible transformations can be used for operations such as:

  Rotation, Scaling, Coordinate transformation, Geometric transformations

  For example, an image can be transformed into another coordinate system and then transformed back.

- **Normalization and Transformations:** Invertible operations allow systems to map data back and forth between different feature spaces or coordinate systems without losing data.

  Normalizing flows use **invertible transformations** to convert a simple probability distribution into a complex data distribution.
  Conceptually:

  Simple distribution → Invertible transformations → Complex data

  And because the transformations are reversible:

  Complex data → Inverse transformations → Simple distribution

  This allows models to perform **generation and inference** while keeping track of the probability density.

- **Data Compression / Representation Learning**

  Generative deep learning models use invertible neural networks to calculate exact likelihoods and generate realistic data samples.

  Invertible neural networks can transform data into another representation while preserving information.

  For example:

  Image → Latent representation

  and then:

  Latent representation → Original image

  This is useful when **information preservation and reconstruction** are important.

Example: If $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ and $X = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Transform: $Y = AX = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}\begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \end{bmatrix}$ Now, $A^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/3 \end{bmatrix}$

Recover the original:

$$X = A^{-1}Y$$
$$= \begin{bmatrix} 1/2 & 0 \\ 0 & 1/3 \end{bmatrix}\begin{bmatrix} 2 \\ 6 \end{bmatrix} = \boxed{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}$$

Example: Find the inverse of the matrix $A = \begin{bmatrix} 4 & 3 & 8 \\ 6 & 2 & 5 \\ 1 & 5 & 9 \end{bmatrix}$ using the formula.

We have, $A = \begin{bmatrix} 4 & 3 & 8 \\ 6 & 2 & 5 \\ 1 & 5 & 9 \end{bmatrix}$

Find the value of determinant of the matrix.

$|A| = 4(18–25) – 3(54–5) + 8(30–2)$

$\Rightarrow |A| = 49$

Find the adjoint of matrix A by computing the cofactors of each element and then getting the cofactor matrix's transpose.

$\text{adj } A = \begin{bmatrix} -7 & 13 & -1 \\ -49 & 28 & 28 \\ 28 & -17 & -10 \end{bmatrix}$

So, the inverse of the matrix is,

$A^{-1} = \frac{1}{49}\begin{bmatrix} -7 & 13 & -1 \\ -49 & 28 & 28 \\ 28 & -17 & -10 \end{bmatrix}$

$\Rightarrow A^{-1} = \begin{bmatrix} -\frac{1}{7} & \frac{13}{49} & -\frac{1}{49} \\ -1 & \frac{4}{7} & \frac{4}{7} \\ \frac{4}{7} & -\frac{17}{49} & -\frac{10}{49} \end{bmatrix}$

Assignments:

X = [0 1 3] and Y = [2 4 0] • Find V = Subspace of X • Find W = Subspace of Y • Describe V intersection W ?

Given $X = \begin{bmatrix} 0 \\ 1 \\ 3 \end{bmatrix}$, $Y = \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}$

We need to find **V = subspace generated by X**, **W = subspace generated by Y**, and $V \cap W$.

1. Subspace V of X

The subspace generated by X contains **all scalar multiples of** X.

$$V = \text{span}\{X\}$$

Therefore, $V = \left\{ a \begin{bmatrix} 0 \\ 1 \\ 3 \end{bmatrix} : a \in \mathbb{R} \right\}$ = $\boxed{V = \left\{ \begin{bmatrix} 0 \\ a \\ 3a \end{bmatrix} : a \in \mathbb{R} \right\}}$

For example: $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \\ 6 \end{bmatrix}, \begin{bmatrix} 0 \\ -1 \\ -3 \end{bmatrix}$ are all in V.

---

2. Subspace W of Y

Similarly, $W = \text{span}\{Y\}$

Therefore, $W = \left\{ b \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix} : b \in \mathbb{R} \right\}$ = $\boxed{W = \left\{ \begin{bmatrix} 2b \\ 4b \\ 0 \end{bmatrix} : b \in \mathbb{R} \right\}}$

For example: $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}, \begin{bmatrix} 4 \\ 8 \\ 0 \end{bmatrix}, \begin{bmatrix} -2 \\ -4 \\ 0 \end{bmatrix}$ are all in W.

---

3. Find $V \cap W$

The **intersection** contains vectors that belong to **both** V **and** W.

Take a vector from V: $v = \begin{bmatrix} 0 \\ a \\ 3a \end{bmatrix}$

and a vector from W: $w = \begin{bmatrix} 2b \\ 4b \\ 0 \end{bmatrix}$

For the vector to be common: $\begin{bmatrix} 0 \\ a \\ 3a \end{bmatrix} = \begin{bmatrix} 2b \\ 4b \\ 0 \end{bmatrix}$

Compare the coordinates: $0 = 2b \Rightarrow b = 0$

Then: $a = 4b = 0, \quad 3a = 0$
So the **only common vector is the zero vector**.

Therefore, $\boxed{V \cap W = \left\{\begin{bmatrix}0\\0\\0\end{bmatrix}\right\}}$

Subspace	Answer
V	span$\{(0,1,3)^T\}$
W	span$\{(2,4,0)^T\}$
$V \cap W$	$\boxed{\{(0,0,0)^T\}}$

Simple interpretation: V and W are two different lines passing through the origin. They have **only the origin in common**, so their intersection is the **zero subspace**.